Adaptive Trust-Based Memory Consolidation for a Self-Improving Coding Assistant

[Author 1 Name]
Computer Engineering
Vishwakarma Institute of Technology, Pune
Pune, India
author1@vit.edu

[Author 2 Name]
Computer Engineering
Vishwakarma Institute of Technology, Pune
Pune, India
author2@vit.edu

[Author 3 Name]
Computer Engineering
Vishwakarma Institute of Technology, Pune
Pune, India
author3@vit.edu

[Author 4 Name]
Computer Engineering
Vishwakarma Institute of Technology, Pune
Pune, India
author4@vit.edu

Abstract— Language-model agents that improve by writing their own memory can also learn wrong lessons and then trust them. This paper presents ATMC (Adaptive Trust-based Memory Consolidation), an outcome-verified memory for a self-improving coding assistant. Every stored note carries a trust score that is set and updated from verified results (unit tests run in a sandbox, generated tests with cross-solution consensus, or the user's verdict) and only for notes the model actually cited. Low-trust notes are blocked, unused notes fade, and similar notes are merged. The assistant also chooses its own practice tasks and tracks which solving strategy works best for each problem type; the model's weights are never changed. With deliberately planted wrong notes on GSM8K and HumanEval/MBPP (3 seeds, 40 tasks per run), ATMC cut the planted notes shown to the model about tenfold (60.0 to 6.0 per run on GSM8K; 61.3 to 6.0 on code), to zero after a three-task warm-up, while pass rates stayed statistically indistinguishable from a no-memory baseline (89.2% vs 88.3% on code). Ablations identify trust scoring as the decisive component. The planner, reflection and curriculum showed no significant accuracy gain, and we report the limitations of this evidence.

Keywords— Self-Evolving Agents, Agent Memory, Trust Scoring, Memory Poisoning, Code Generation, Large Language Models

I. Introduction

Language-model agents increasingly improve by keeping notes about their own attempts and reading them back on later tasks [1]–[5]. Unlike curated knowledge, these notes are written by a fallible model. A wrong note, whether it comes from a hallucination, a weak test or a deliberate attack [6], [7], is retrieved like any other and can steer later answers. Most memory designs focus on what to store and how to retrieve it; few ask whether a stored note deserves to be trusted.

This work addresses that gap. ATMC treats memory as something that is earned: a note's trust rises or falls with verified outcomes, notes below a threshold are never retrieved, unused notes fade, and similar notes are merged to keep the memory small. We build the memory into a complete coding assistant, with a sandboxed checker, a self-directed practice loop and a chat interface, and evaluate it under controlled corruption of its memory.

Key Contributions:

- An outcome-verified, trust-scored memory (ATMC) with citation-conditioned trust updates, trust-gated ranking, adaptive forgetting and compression; it is training-free and independent of the underlying model.
- A verification layer for tasks with and without tests: sandboxed execution, auto-verification with generated tests and cross-solution consensus, evidence-weighted learning, and a dispute path.
- Self-directed improvement: a weakness-targeted practice curriculum and a champion/challenger strategy selector with automatic rollback.
- A reproducible evaluation with paired statistics on GSM8K, HumanEval and MBPP, reporting negative results as well as positive ones.

II. Related Work

Memory for language-model agents. Reflexion stores verbal reflections after failures and conditions later attempts on them [1]. Generative Agents retrieve memories by recency, importance and relevance and add higher-level reflections [2]. MemGPT pages information between context tiers [3], A-MEM organizes notes as an evolving linked memory [4], and Voyager accumulates a library of reusable skills [5]. These systems decide what to keep and fetch; ATMC adds a score for how far a note has been shown to help, and removes notes that fail that test.

Memory poisoning. PoisonedRAG shows that a handful of malicious passages can steer a retrieval-augmented model, and reports that existing defenses are insufficient [6]. AgentPoison poisons an agent's long-term memory or

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

knowledge base with a backdoor trigger [7]. ATMC targets the same surface from the defender's side, using outcome-based trust.

Self-improvement and verification. Self-Refine revises an answer using the model's own feedback [8]. CodeT has the model write tests and picks solutions by agreement with them [9]; we use the same agreement idea, but to gate what enters memory. A recent survey organizes self-evolving agents by what evolves (models, memory, tools, architecture) and when [10]; ATMC operates at the memory and strategy-selection level and leaves the weights untouched.

Benchmarks and components. We evaluate on HumanEval [11], MBPP [12] and GSM8K [13]; relevance is measured with BGE sentence embeddings [14] in a retrieval-augmented setting [15].

III. Methodology

The system is a pipeline in which each stage has one job: the interface collects a question, the backend plans, retrieves memory, calls the language model and checks the result in a sandbox, and the memory is updated from the outcome. The components are modular, so the model, the memory or the interface can be replaced independently. The language roles (planner, solver, reflector, summarizer, test writer) are separate prompts to one model run in sequence; they are modules of a single pipeline and not independent, communicating agents.

Table 1: System components and technologies

Component	Technology / Tool	Purpose
Chat interface	React + Vite	Questions, answers, memory view
Backend API	FastAPI (Python)	Agent loop, persistence
Language model	Gemini API	Reasoning, tests, summaries
Embeddings	BGE-small (fastembed)	Note and task similarity
Memory (ATMC)	Own implementation	Trust, forgetting, merging
Checker	Python sandbox	Run tests safely
Storage	SQLite	Notes, history, evidence
Evaluation	Own harness + pytest	Paired statistics

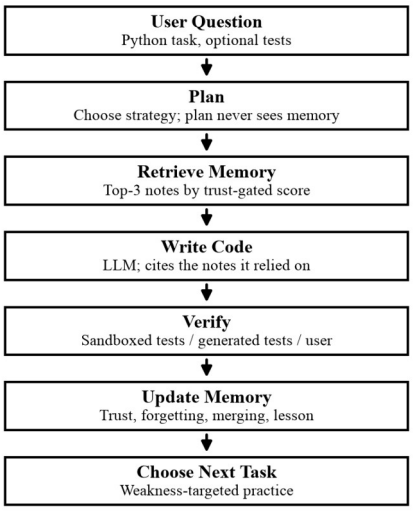


Figure 1: End-to-end workflow of the assistant for one question

A. System Architecture

The assistant has four parts (Fig. 2): a web interface, a backend that runs the agent loop, the ATMC memory with its SQLite store, and a sandbox that executes generated code. Retrieval embeddings are computed locally; only the language model is an external service. The implementation is about 3,392 lines of Python, 1,842 lines of JavaScript and CSS, and 1,636 lines of tests (148 automated tests).

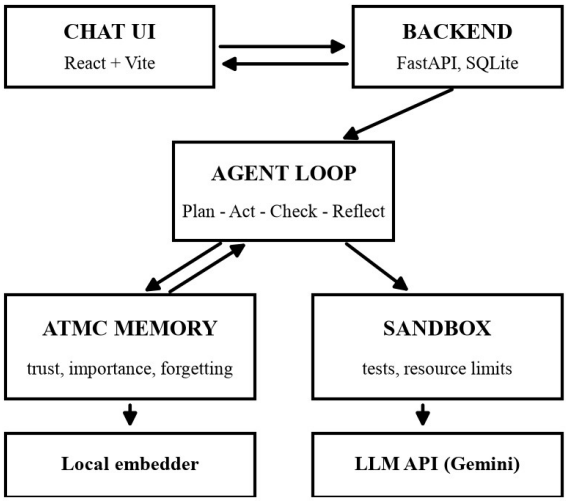


Figure 2: System architecture

B. ATMC Memory Model

A note stores the task, the attempted solution, its outcome and any lesson, together with a trust score, a use count, the evidence behind its outcome (tests, user, generated tests, planted, merged) and counts of verified successes and failures when it was relied on. Trust is initialized from the checker's verdict and then updated, but only for notes the model reports having used, which avoids crediting or blaming notes that were merely retrieved:

$$T' = \mathrm{clip}(T + w\eta\,[\mathrm{ok}] - w\delta\,[\mathrm{fail}],\ 0,\ 1) \qquad (1)$$

where [ok] and [fail] mark a verified correct or wrong outcome, with $\eta = 0.05$, $\delta = 0.15$ ($\delta > \eta$ so bad notes lose

trust faster than good ones gain it) and an evidence weight $w = 1$ for tests and user verdicts and 0.5 for generated tests. Notes with trust below $T_{min} = 0.30$ are excluded from retrieval. Relevance to a query q is the cosine similarity of normalized embeddings, weighted by how often a note has been used:

$$I(m) = \cos(e_q, e_m) \cdot (1 + \log(1 + h_m)) \quad (2)$$

Notes also fade with the number of task steps since creation, following an exponential forgetting curve [19], more slowly when they are trusted and important:

$$F = \exp(-\lambda \Delta t), \quad \lambda = \lambda_0 / (1 + \alpha T + \beta I) \quad (3)$$

Retrieval ranks notes by a composite in which trust also gates importance, so a barely-trusted note cannot enter the top $k = 3$ merely by looking similar:

$$S = 0.4\ T + 0.4\ I \cdot T + 0.2\ F \quad (4)$$

C. Verification and Learning Signals

When a task has tests (HumanEval, MBPP, or assertions the user supplies), the answer runs in a sandbox that denies network access and writes outside a temporary directory, hides credentials, and enforces a timeout and resource limits. The sandbox is not a hard boundary against hostile code. When a user task has no tests, auto-verification asks the model for up to eight assert statements and for two independent alternative solutions. A generated test counts as reliable only if at least two solutions pass it, and a verdict requires a body of agreed tests (at least two and half of those generated); otherwise the result is inconclusive. A wrong first answer is replaced by an alternative that passes the reliable tests, and a request too vague to test is declared underspecified. The check measures consistency with the request as the model understood it, not the user's unstated intent, so generated-test evidence is given half weight and the user can dispute the result, which retires the note.

D. Forgetting, Merging and Pinning

Every ten additions, notes whose trust is below the gate, or whose retention is below 0.05, are removed. Every eight additions, the three oldest, least-used notes are merged into a model-written summary, which records its lineage (summed evidence, version number). Pinned notes are exempt from both, and imported notes restart at the initial trust so a file cannot smuggle in a highly trusted wrong note.

E. Practice and Strategy Selection

The assistant picks its own practice tasks: each unattempted task is scored by its similarity to the assistant's own failure notes, with a 20% chance of choosing at random to avoid fixating. It also tracks three solving strategies (plan first, direct, and write-then-review) for each problem type, found by nearest-prototype matching with the same embedder. The plan-first default is replaced for a problem type only when another strategy has at least five attempts and the lower bound of its Wilson interval [16] reaches the default's success rate; the choice is recomputed from the evidence each time, so a strategy that starts failing is rolled back automatically. Exploration happens during practice, where outcomes come from real tests.

F. Experimental Setup

Datasets. GSM8K [13] (test split), HumanEval [11] and MBPP [12] (four MBPP tasks whose reference solutions fail their own tests were excluded). Each run processes a seeded sample of 40 tasks (seeds 42, 123, 456; the GSM8K ablation also uses 789 and 1011). The model is gemini-3.5-flash-lite [20] at temperature 0; embeddings are BAAI/bge-small-en-v1.5 (384 dimensions). Memory settings: top-3 retrieval, initial trust 0.5, merging every 8 additions.

Corruption. Every third task (13–14 per run) receives a planted wrong note for that task: a wrong numeric answer (GSM8K) or a single-site mutation of the reference solution verified to fail its tests (code). These notes are recorded as failed attempts. Baselines: no memory; static memory (stores and retrieves everything by similarity); a Reflexion-style buffer of the three most recent failure reflections [1]; and ATMC.

Metrics and statistics. Pass rate; planted notes shown to the model, in total and after the first three tasks (before which no genuine note can exist); planted notes the model cited; tokens per task; and retrieval latency. Conditions are compared on the same tasks with a paired bootstrap 95% confidence interval [18] and the exact McNemar test [17]. Agent components are also scored on 20 unseen tasks per seed with memory frozen.

IV. Results and Discussion

A. Resistance to Planted Wrong Notes

Table 2: Pass rate and exposure to planted notes (3 seeds, 40 tasks per run). Shown = planted notes in the model's context per run (after warm-up in parentheses); Relied = planted notes the model cited; floor = optional minimum-similarity filter (0.75)

Condition	Pass rate (%)	Shown	Relied	Tokens
GSM8K: static	98.3 ± 1.4	60.0 (51.0)	5.0	614
GSM8K: ATMC	100.0 ± 0.0	6.0 (0.0)	0	828
Code: no memory	89.2 ± 5.8	0.0 (0.0)	0	104
Code: static	89.2 ± 5.8	61.3 (52.7)	9.3	638
Code: Reflexion-style	89.2 ± 6.3	53.3 (44.3)	0.3	736
Code: ATMC	88.3 ± 7.2	6.0 (0.0)	0	624
Code: ATMC + floor	88.3 ± 5.2	8.3 (7.7)	1.3	342

Table 2 and Fig. 3 show the central result. On GSM8K, static memory placed 60.0 planted notes in front of the model per run and the model cited 5.0 of them; ATMC showed 6.0 and none was cited. On code, static memory showed 61.3 and the Reflexion-style buffer 53.3, against 6.0 for ATMC, a roughly tenfold reduction. The remaining 6 exposures per run are a cold-start floor: in the first three tasks the planted notes are the only notes in memory, so nothing can distinguish them yet; after that, ATMC showed none.

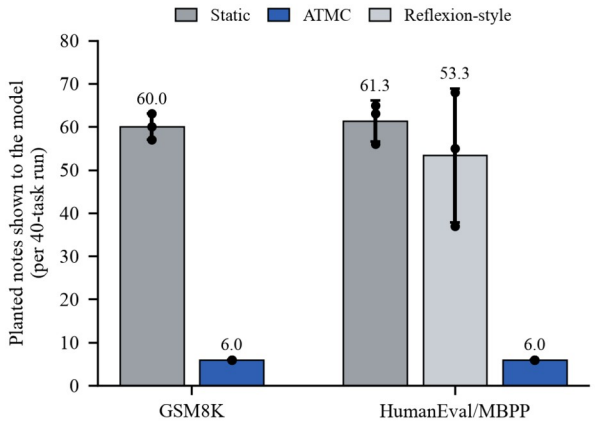


Figure 3: Planted notes shown to the model per 40-task run (mean ± s.d.; dots are seeds)

Accuracy did not separate the systems. On GSM8K ATMC scored 100.0% against 98.3% (difference +1.7 points, 95% CI [+0.0, +4.2], McNemar p = 0.50). On code every paired comparison against no memory had p = 1.0 (no memory 89.2%, static 89.2%, ATMC 88.3%). The base model already solves about 90% of these tasks alone, so the measurable benefit here is protection from corrupted notes, not higher accuracy.

B. Ablation

Table 3: Ablation: planted notes in context after warm-up (mean ± s.d.), notes kept, and pass rate (GSM8K / code)

Variant	GSM8K (5 seeds)	Code (3 seeds)	Notes	Pass %
Full ATMC	0.0 ± 0.0	0.0 ± 0.0	42	97.5/88.3
w/o Trust	111.0 ± 0.0	111.0 ± 0.0	42	97.5/89.2
w/o Forgetting	14.8 ± 20.3	26.0 ± 19.1	42	97.0/89.2
w/o Merging	0.0 ± 0.0	0.0 ± 0.0	54	97.0/88.3

Removing trust scoring let 111 planted notes through per run on every seed in both domains, against zero for the full system (Fig. 4), because without trust every note has equal weight and only similarity ranks them. Removing forgetting had a smaller, inconsistent effect (14.8 ± 20.3 on GSM8K, 26.0 ± 19.1 on code); at these run lengths forgetting mostly acts by lowering ranks, and deletion needs roughly 70 steps. Merging reduced the stored notes from 54 to 42 (22%) with no change in exposure. No ablation changed the pass rate (all p = 1.0).

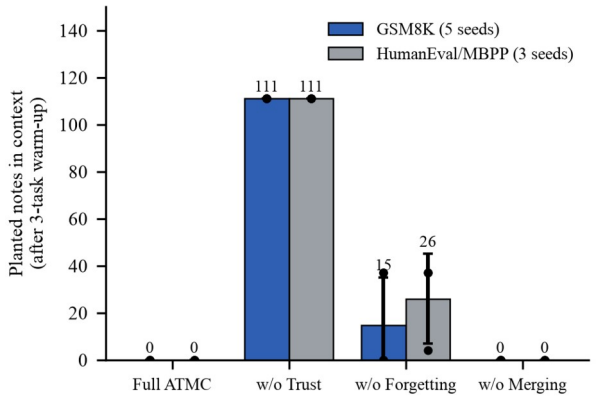


Figure 4: Ablation: planted notes in context after the three-task warm-up (mean ± s.d.; dots are seeds)

C. Trust Dynamics

Fig. 5 is computed from the update rule itself. A note that has passed the gate once sits at 0.55; after two verified failures in which it was cited it falls below the retrieval gate, and after four if its evidence came from generated tests (half weight). A static memory keeps it forever. In our experiments the planted notes enter as failed attempts, so they start at 0.35 against 0.55 for genuine notes and are outranked from the fourth task onwards; the learned update path matters most when a wrong note is mistakenly recorded as correct, which we did not test (Section V).

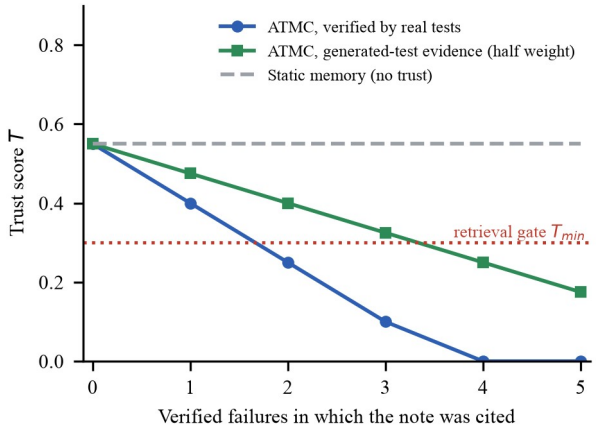


Figure 5: Trust of a note over verified failures in which it is cited (from the update rule)

D. Planner, Reflection and Curriculum

Table 4: Agent components on 20 held-out tasks per seed (3 seeds): change versus ATMC alone in points with 95% CI, McNemar p, tokens per task

Condition	Held-out (%)	Δ vs ATMC [95% CI]	p	Tokens
ATMC alone	88.3 ± 2.9	-	-	1164
+ Planner	95.0 ± 5.0	+6.7 [+1.7, +13.3]	0.125	1526
+ Reflection	88.3 ± 2.9	+0.0 [+0.0, +0.0]	1.000	1193
+ Planner + Reflection	95.0 ± 5.0	+6.7 [+1.7, +13.3]	0.125	1561
Random task choice	90.0 ± 0.0	+1.7 [+0.0, +5.0]	1.000	1178
Weakness-targeted choice	90.0 ± 0.0	+1.7 [-3.3, +6.7]	1.000	900

No component showed a statistically significant benefit (Table 4, Fig. 6). The planner is the only lead: +6.7 points on held-out tasks, better on 4 tasks and worse on 0, with a bootstrap interval above zero but an exact McNemar p = 0.125, because only 4 tasks changed; it also costs 31% more tokens. Reflection changed nothing: it runs only after a failure, and failures are rare at this baseline. Neither curriculum beat plain ATMC. A larger held-out set would be needed to confirm or reject the planner effect.

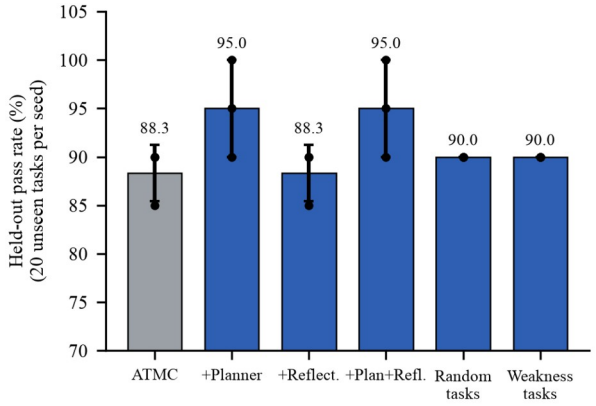


Figure 6: Held-out pass rate of the agent components (mean ± s.d.; dots are seeds)

E. Cost and Overhead

ATMC used 624 prompt tokens per task, about the same as static memory (638) and six times the no-memory baseline (104); the prompt always carries the top three notes, so it does not grow with memory size, while merging bounds the store itself. Retrieval took about 1.2 ms per query. An optional relevance floor (minimum similarity 0.75) halved tokens to 342 but let 7.7 planted notes through after warm-up, and it did not recover accuracy, so it is off by default. Auto-verification costs about three extra model calls per question; we exercised it on a handful of requests (verified, inconclusive and underspecified outcomes all occurred) but did not evaluate it statistically.

V. Limitations

The evidence supports protection, not improvement. The base model already solves about 90% of the tasks without memory, so differences in accuracy cannot be detected, and in the static code runs the model cited a planted note on 28 steps yet failed on only one of them. The planted notes were recorded as failed attempts, so ATMC started them at lower trust; a stronger test, which we have not run, inserts wrong notes recorded as correct. A wrong note that is cited in a correct answer still gains trust. Runs use 40 tasks and 3 seeds (5 for one ablation) on one model family, in Python only, and differences of a few points are within run-to-run noise at temperature 0. Auto-verification checks consistency, not intent, and strategy selection was exercised only on a small number of practice attempts, so it is a working mechanism and not a measured gain. The sandbox is not a hard security boundary, the free API tier limited the number of seeds, and the roles are modules of one pipeline rather than independent agents.

VI. Conclusion

We presented ATMC, an outcome-verified memory for a self-improving coding assistant, and the system built around it. Trust scoring kept planted wrong notes out of the model's context roughly tenfold (61 to 6 per run on code) without a measurable cost in accuracy, and the ablations identify trust, not forgetting or merging, as the decisive component. The assistant also verifies its own answers, practises its weak spots and tracks which strategy works, but these additions have not yet been shown to raise accuracy.

Future work includes harder benchmarks or weaker models where memory errors change accuracy, wrong notes recorded as correct, a second model family, shared memory across cooperating agents, and an indexed store for larger memories. Treating an agent's memory as untrusted input that must earn its influence is a simple principle that fits any agent that writes its own notes.

Acknowledgment

The authors would like to thank the faculty and mentors at Vishwakarma Institute of Technology, Pune, in particular our guide, Prof. Kirti Wanjale, for their guidance and support throughout this work.

References

[1] N. Shinn et al., “Reflexion: Language agents with verbal reinforcement learning,” arXiv:2303.11366, 2023.

[2] J. S. Park et al., “Generative agents: Interactive simulacra of human behavior,” in Proc. ACM UIST, 2023.

[3] C. Packer et al., “MemGPT: Towards LLMs as operating systems,” arXiv:2310.08560, 2023.

[4] W. Xu et al., “A-Mem: Agentic memory for LLM agents,” arXiv:2502.12110, 2025.

[5] G. Wang et al., “Voyager: An open-ended embodied agent with large language models,” arXiv:2305.16291, 2023.

[6] W. Zou, R. Geng, B. Wang, and J. Jia, “PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models,” in Proc. USENIX Security, 2025.

[7] Z. Chen et al., “AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases,” in Proc. NeurIPS, 2024.

[8] A. Madaan et al., “Self-Refine: Iterative refinement with self-feedback,” arXiv:2303.17651, 2023.

[9] B. Chen et al., “CodeT: Code generation with generated tests,” in Proc. ICLR, 2023 (arXiv:2207.10397).

[10] “A survey of self-evolving agents,” arXiv:2507.21046, 2025.

[11] M. Chen et al., “Evaluating large language models trained on code,” arXiv:2107.03374, 2021.

[12] J. Austin et al., “Program synthesis with large language models,” arXiv:2108.07732, 2021.

[13] K. Cobbe et al., “Training verifiers to solve math word problems,” arXiv:2110.14168, 2021.

[14] S. Xiao et al., “C-Pack: Packed resources for general Chinese embeddings,” in Proc. ACM SIGIR, 2024 (arXiv:2309.07597).

[15] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” arXiv:2005.11401, 2020.

[16] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” J. Amer. Statist. Assoc., vol. 22, no. 158, 1927.

[17] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” Psychometrika, vol. 12, no. 2, 1947.

[18] B. Efron, “Bootstrap methods: Another look at the jackknife,” Ann. Statist., vol. 7, no. 1, 1979.

[19] H. Ebbinghaus, Memory: A Contribution to Experimental Psychology. New York, NY, USA: Teachers College, Columbia Univ., 1913.

[20] Google, “Gemini API documentation,” ai.google.dev, accessed 2026.